Hamburg University of Technology
Institute of Mathematics
Chair Numerical Mathematics

Jens-Peter M. Zemke
Finn Sommer
Lars Stietz

Advanced Machine Learning
Winter Semester 2025/2026
Exercise Sheet 6
Week 7, 24.11.2025 – 28.11.2025

Objectives of this exercise sheet:

- Exercise 1 is somewhat mathematical, but actually quite simple and tells you why we use convolution (like Julia does) and not the cross-correlation (like TensorFlow and PyTorch). It opens up the way to use other means to implement fast convolutions.
- Exercise 2 is a little bit more involved, but if you follow the steps of the lecture for the 1D case it is nevertheless doable. The main goal is to ensure that you are familiar with the feedforward and backprop of 2D convolutions and their relation to (block) Toeplitz matrices. It also deepens your understanding of backprop for convolutional layers.
- Exercise 3 highlights two important things: on the one hand the differences and similarities between dense and convolutional layers, on the other hand the intricate details of the NCHW format and the way of computation of convolutions with 4-tensors. After having worked on this exercise you will understand and remember these things.
- Exercise 4 is twofold: part a) is to understand how TensorFlow (and any similar libraries) implement "convolutions" in 2D, part b) is for the fun of it, as you simply can play around with one parameter to get a feeling for a working program.

Homework:

Exercise 1: (2 points) level •○○ relevance •○○
Let $\mathbf{i} = (i_1, \ldots, i_d)$ be a multi-index. Prove that multi-dimensional (full) convolution

$$(\mathbf{X} * \mathbf{F})(n_1, \ldots, n_d) = \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \ldots, n_d - i_d) \cdot \mathbf{F}(i_1, \ldots, i_d)$$

is commutative, i.e., $\mathbf{X} * \mathbf{F} = \mathbf{F} * \mathbf{X}$.

Exercise 2: (1+4 points) level ••○ relevance •••
In Lecture 4 we showed, based on the interpretation in terms of Toeplitz matrices, that in 1D

$$\frac{\partial C}{\partial \mathbf{f}} = \widetilde{\mathbf{a}} * \boldsymbol{\delta} = \widetilde{(\mathbf{a} * \widetilde{\boldsymbol{\delta}})},$$

where the tilde represents a flipping of the entries. Suppose that in 2D we have a kernel $\mathbf{F} \in \mathbb{R}^{2\times 2}$ with input $\mathbf{A} \in \mathbb{R}^{3\times 3}$. The convolution $\mathbf{A} * \mathbf{F}$ is then given by

$$\begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix},$$

or, in vector notation, by

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ \hline y_{2,1} \\ y_{2,2} \end{pmatrix} = \left(\begin{array}{ccc|ccc|ccc} f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} \end{array}\right) \begin{pmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \\ \hline a_{2,1} \\ a_{2,2} \\ a_{2,3} \\ \hline a_{3,1} \\ a_{3,2} \\ a_{3,3} \end{pmatrix}.$$

a) How does $\mathbf{\Delta}$ from the backpropagation look like in this case?
b) Prove the equality

$$\frac{\partial C}{\partial \mathbf{F}} = \widetilde{\mathbf{A}} * \mathbf{\Delta}$$

using an approach corresponding to the 1D case, based on block Toeplitz matrices.

Exercises (in class):

Exercise 3: (2+5 points) level ●●● relevance ●●●

a) Count the number of parameters layer-wise and in total for the two following network parts (see also Figure 1). Every layer has a bias.
   (i) The input are RBG images of size 10×10, followed by two convolutional layers, the first with 3 filters of size 5×5, the second with 4 filters of size 3×3.
   (ii) The input are vectors of size 300, followed by two dense layers, the first with 108 neurons, the second with 64 neurons.
b) Write convolutional layers as dense layers and vice versa:
   (i) Write the second convolutional layer from part a) as dense layer. Hint: you have to find a weight matrix $\mathbf{W}$ and a bias $\mathbf{b}$ that maps the vectorized input to the vectorized affine linear combination.
   (ii) Write the first dense layer as a convolutional layer. How many filters do you have? Which size do these filters have? Hint: there are different ways to achieve this.

Exercise 4: (2+2 points) level ●●○ relevance ●●●

a) In the skeleton `exercise4a_skel.py` one `tf.keras.layers.Conv2D` is generated, called, and the weights are extracted. Define the function `my_tfconv` in the skeleton such that (up to round-off) the same results are computed. How are the filters stored? Does TensorFlow/Keras use convolution or cross-correlation?
b) In `exercise4b_skel.py` you can find functions generating a FNN (see Table 1) and a CNN (see Table 2) network in TensorFlow/Keras for the Fashion MNIST dataset. Both networks use two dropout layers.
   First train the networks without any dropout, i.e., the probabilities set to `0`. Then try different dropout probabilities. What do you observe?

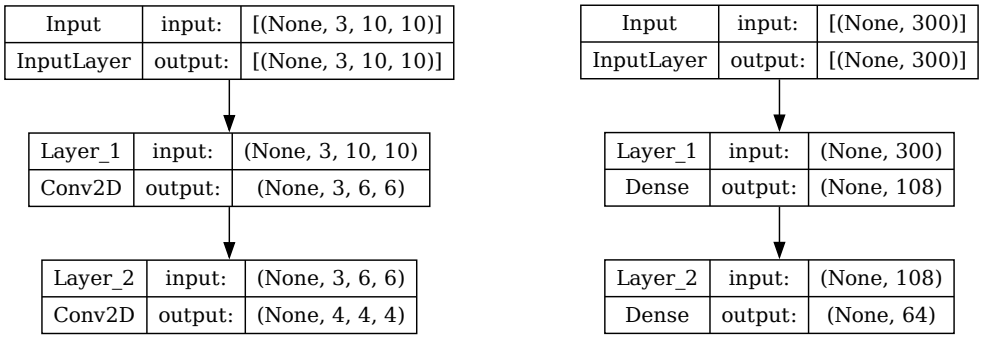

Figure 1: Parts of a convolutional network with 10×10 RGB images as input (left) and of a dense network with vectors with 300 entries as inputs (right).

Layer (type)	Output Shape	# Parameters
fnn_dense_1 (Dense)	(None, 256)	200,960
fnn_dropout_1 (Dropout)	(None, 256)	0
fnn_dense_1 (Dense)	(None, 128)	32,896
fnn_dropout_2 (Dropout)	(None, 128)	0
fnn_output (Dense)	(None, 10)	1290

Table 1: TensorFlow summary of the FNN.

Layer (type)	Output Shape	# Parameters
cnn_conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
cnn_max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
cnn_conv2d_2 (Conv2D)	(None, 11, 11, 64)	18,496
cnn_max_pooling2d_2	(None, 5, 5, 64)	0
cnn_dropout_1 (Dropout)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
cnn_dense_1 (Dense)	(None, 128)	204,928
cnn_dropout_2 (Dropout)	(None, 128)	0
cnn_output (Dense)	(None, 10)	1,290

Table 2: TensorFlow summary of the CNN.